AI Agent Coding for GitLab — Design Document

> ***Version:*** *1.0*
> ***Date:*** *2026-03-06*
> ***Author:*** *AI-Generated*
> ***Status:*** *Draft*
> ***Source:*** *requirement-agent-coding.md*

Table of Contents

1. Tổng quan (Overview)
2. Kiến trúc hệ thống (System Architecture)
3. Thành phần hệ thống (System Components)
    - 3.4 Multi-Repository Manager
4. Workflow tổng thể (End-to-End Workflow)
5. Workflow chi tiết từng phase
    - 5.1 Phase 1 — Init
        - 5.1.3 HTML UI Mockup Generation
    - 5.2 Phase 2 — Implement
    - 5.3 Phase 3 — Review
    - 5.4 Phase 4 — Done
6. Sequence Diagrams
7. State Machine
8. Data Model
9. GitLab API Integration
10. Tech Stack & Dependencies
11. Error Handling & Edge Cases
12. Security Considerations
13. Docker Deployment

---

1. Tổng quan (Overview)

1.1 Mục tiêu

Xây dựng một hệ thống **AI Agent** tích hợp với **self-hosted GitLab**, có khả năng:

- **Tự động lên kế hoạch** từ file requirement
- **Tạo và quản lý issues** trên GitLab
- **Tự động implement code** theo từng issue
- **Tương tác với user** thông qua comments trên GitLab
- **Tạo merge request** khi hoàn thành và xử lý review feedback
- **Đóng issues** sau khi merge thành công
- **Tự động tạo HTML UI Mockup** từ requirement để user review trước khi implement
- **Quản lý nhiều git repository** trong cùng một thư mục code (monorepo hoặc multi-service)

1.2 Phạm vi

Aspect	Detail

AI Engine	Claude Code
Source Control	Self-hosted GitLab
Input	Requirement document (markdown)
Output	Code, Documents, Issues, Merge Requests
Interaction	GitLab Issues & MR comments

---

2. Kiến trúc hệ thống (System Architecture)

```
graph TB
    subgraph "User Layer"
        U["👤 User / Project Manager"]
    end

    subgraph "GitLab Layer"
        GL["🦊 Self-hosted GitLab"]
        GL_ISSUES["📋 Issues"]
        GL_MR["🔀 Merge Requests"]
        GL_REPO["📁 Repository"]
        GL_HOOKS["🪝 Webhooks"]
        GL_COMMENTS["💬 Comments"]

        GL --> GL_ISSUES
        GL --> GL_MR
        GL --> GL_REPO
        GL --> GL_HOOKS
        GL --> GL_COMMENTS
    end

    subgraph "Orchestrator Layer"
        ORCH["🎯 Orchestrator Service"]
        HOOK_SERVER["🌐 Webhook Server"]
        SCHEDULER["⏰ Task Scheduler"]
        STATE_MGR["📊 State Manager"]
        REPO_MGR["📦 Multi-repo Manager"]

        ORCH --> HOOK_SERVER
        ORCH --> SCHEDULER
        ORCH --> STATE_MGR
        ORCH --> REPO_MGR
    end

    subgraph "AI Agent Layer"
        AGENT["🤖 Claude Code Agent"]
        PLANNER["📝 Planner Module"]
        CODER["💻 Coder Module"]
        REVIEWER["🔍 Review Handler"]
        DOC_GEN["📄 Document Generator"]
```

```
        MOCKUP["🎨 Mockup Generator"]

        AGENT --> PLANNER
        AGENT --> CODER
        AGENT --> REVIEWER
        AGENT --> DOC_GEN
        AGENT --> MOCKUP
    end

    subgraph "Storage Layer"
        LOCAL_FS["💾 Local File System"]
        CONFIG["⚙️ Configuration"]
    end

    U -->|"1. Provide requirement"| GL_REPO
    U -->|"2. Review & comment"| GL_ISSUES
    U -->|"3. Review MR"| GL_MR

    GL_HOOKS -->|"Events"| HOOK_SERVER
    ORCH -->|"GitLab API"| GL
    AGENT -->|"Git operations"| GL_REPO
    AGENT -->|"Read/Write"| LOCAL_FS
    ORCH -->|"Invoke"| AGENT
    CONFIG -->|"Settings"| ORCH
```

---

3. Thành phần hệ thống (System Components)

3.1 Orchestrator Service

Thành phần trung tâm điều phối toàn bộ quy trình:

Component	Responsibility
Webhook Server	Nhận events từ GitLab (comment, issue update, MR events)
Task Scheduler	Lên lịch và quản lý thứ tự thực thi các tasks
State Manager	Theo dõi trạng thái của từng issue, phase hiện tại
GitLab Client	Giao tiếp với GitLab API
Multi-repo Manager	Quản lý nhiều git repositories, routing tasks đến đúng repo

3.2 AI Agent (Claude Code)

Module	Responsibility
Planner	Phân tích requirement → tạo plan, chia nhỏ thành issues
Document Generator	Sinh Architecture, DB Schema, API Docs, Test Cases
Mockup Generator	Sinh HTML/CSS/JS UI mockup từ requirement để user review

Coder	Implement code theo từng issue
Review Handler	Xử lý feedback từ user comments, update code/docs

3.3 GitLab Integration

```
graph LR
    subgraph "GitLab API Endpoints"
        A["Projects API"]
        B["Issues API"]
        C["Merge Requests API"]
        D["Repository API"]
        E["Notes API - Comments"]
        F["Webhooks API"]
    end

    subgraph "Operations"
        A1["Get project info"]
        B1["Create / Update / Close issues"]
        C1["Create / Update MR"]
        D1["Push commits, branches"]
        E1["Read / Write comments"]
        F1["Register webhook listeners"]
    end

    A --> A1
    B --> B1
    C --> C1
    D --> D1
    E --> E1
    F --> F1
```

3.4 Multi-Repository Manager

Cho phép agent làm việc với **nhiều git repositories** trong cùng một thư mục code, phù hợp với kiến trúc microservices hoặc monorepo có nhiều sub-project.

```
graph TB
    subgraph "Working Directory"
        ROOT["📁 /workspace"]
        R1["📁 repo-frontend"]
        R2["📁 repo-backend"]
        R3["📁 repo-infra"]
        ROOT --> R1
        ROOT --> R2
        ROOT --> R3
    end

    subgraph "Multi-repo Manager"
        DETECT["🔍 Repo Detector"]
```

```
        ROUTER["🔀 Task Router"]
        SYNC["🔄 Branch Sync"]
        CTX["📋 Context Switcher"]

        DETECT --> ROUTER
        ROUTER --> SYNC
        ROUTER --> CTX
    end

    DETECT -->|"scan"| ROOT
    ROUTER -->|"route task"| R1
    ROUTER -->|"route task"| R2
    ROUTER -->|"route task"| R3
```

Feature	Detail
Auto-detection	Tự động phát hiện tất cả git repos trong working directory
Task Routing	Phân tích issue → xác định repo nào cần thay đổi
Cross-repo Issues	Một issue có thể span nhiều repos (e.g., API + Frontend)
Branch Isolation	Mỗi repo có feature branch riêng, đồng bộ cùng naming convention
Context Switching	Agent tự động switch context khi làm việc với repo khác
Multi-MR	Tạo MR riêng trên từng GitLab project, link lẫn nhau

---

4. Workflow tổng thể (End-to-End Workflow)

4.1 High-Level Flow

```
flowchart TD
    START(["🚀 Start"]) --> INPUT["📄 Nhận Requirement File"]
    INPUT --> INIT

    subgraph INIT ["Phase 1: INIT"]
        direction TB
        I1["Phân tích requirement"] --> I2["Generate documents"]
        I2 --> I3["Tạo issues trên GitLab"]
        I3 --> I4["User review & feedback"]
        I4 --> I5{User approve?}
        I5 -->|No| I6["Update documents & issues"]
        I6 --> I4
        I5 -->|Yes| I7["✅ Plan confirmed"]
    end

    INIT --> IMPL

    subgraph IMPL ["Phase 2: IMPLEMENT"]
        direction TB
```

```
        P1["Chọn issue tiếp theo"] --> P2["Tạo feature branch"]
        P2 --> P3["AI implement code"]
        P3 --> P4["Commit & push"]
        P4 --> P5["Update issue status"]
        P5 --> P6{User comment?}
        P6 -->|Yes| P7["Xử lý feedback"]
        P7 --> P3
        P6 -->|No| P8{Còn issue?}
        P8 -->|Yes| P1
        P8 -->|No| P9["✅ All issues done"]
    end

    IMPL --> REVIEW

    subgraph REVIEW ["Phase 3: REVIEW"]
        direction TB
        R1["Tạo Merge Request"] --> R2["User review MR"]
        R2 --> R3{MR approved?}
        R3 -->|No| R4["AI xử lý review comments"]
        R4 --> R5["Update code & push"]
        R5 --> R2
        R3 -->|Yes| R6["✅ MR approved"]
    end

    REVIEW --> DONE

    subgraph DONE ["Phase 4: DONE"]
        direction TB
        D1["Merge MR vào main"] --> D2["Đóng tất cả issues"]
        D2 --> D3["Cleanup branches"]
        D3 --> D4["📊 Generate report"]
    end

    DONE --> END(["🏁 Complete"])

    style INIT fill:#1a1a2e,stroke:#e94560,color:#fff
    style IMPL fill:#1a1a2e,stroke:#0f3460,color:#fff
    style REVIEW fill:#1a1a2e,stroke:#533483,color:#fff
    style DONE fill:#1a1a2e,stroke:#16c79a,color:#fff
```

---

5. Workflow chi tiết từng Phase

5.1 Phase 1 — Init

> ***Mục tiêu:*** *Từ requirement file, AI Agent tự động sinh documents, tạo issues, và chờ user review.*

```
flowchart TD
    A(["📩 Nhận Requirement"]) --> B["Parse requirement file"]
    B --> C["Phân tích scope & features"]
    C --> D["Xác định modules & components"]
```

```
D --> GEN
subgraph GEN ["📄 Document Generation"]
    direction TB
    G1["🏗️ Generate Architecture Doc"]
    G2["🗃️ Generate Database Schema"]
    G3["🔌 Generate API Documentation"]
    G4["🧪 Generate Test Cases"]
    G5["📋 Generate Implementation Plan"]
    G6["🎨 Generate HTML UI Mockup"]

    G1 --> G2 --> G3 --> G4 --> G5 --> G6
end

GEN --> H["Tạo branch: docs/init-plan"]
H --> I["Commit documents vào repo"]
I --> J["Tạo issues trên GitLab"]

J --> ISSUES
subgraph ISSUES ["📋 Issue Creation"]
    direction TB
    IS1["Issue #1: Setup project structure"]
    IS2["Issue #2: Database implementation"]
    IS3["Issue #3: API endpoints"]
    IS4["Issue #N: ..."]

    IS1 --- IS2 --- IS3 --- IS4
end

ISSUES --> K["Gán labels, milestones, assignees"]
K --> L["📢 Notify user to review"]

L --> M{User review}
M -->|"Comment: Cần thay đổi"| N["AI đọc comment"]
N --> O["Update documents"]
O --> P["Update issues"]
P --> Q["Commit changes"]
Q --> L

M -->|"Approve: LGTM ✅"| R(["✅ Phase 1 Complete"])

style GEN fill:#0d1117,stroke:#58a6ff,color:#c9d1d9
style ISSUES fill:#0d1117,stroke:#f78166,color:#c9d1d9
```

5.1.1 Chi tiết Document Generation

Document	Nội dung	Format
Architecture	System overview, component diagram, tech decisions	Markdown + Mermaid
Database Schema	Tables, relationships, indexes, migrations	Markdown + SQL

API Documentation	Endpoints, request/response, authentication	OpenAPI / Markdown
Test Cases	Unit tests, integration tests, E2E scenarios	Markdown
Plan	Phased implementation, dependencies, timeline	Markdown + Gantt
HTML UI Mockup	Interactive HTML prototype với full UI screens, navigation, placeholder data	HTML + CSS + JS

5.1.2 Issue Structure

Mỗi issue được tạo với cấu trúc:

```
## 📋 Issue Title: [Feature/Task Name]

### Description
[Mô tả chi tiết task cần thực hiện]

### Acceptance Criteria
- [ ] Criteria 1
- [ ] Criteria 2
- [ ] Criteria 3

### Technical Notes
[Chi tiết kỹ thuật, references đến architecture/API docs]

### Dependencies
- Depends on: #issue_number
- Blocks: #issue_number

### Labels
`phase:implement` `priority:high` `component:api`
```

5.1.3 HTML UI Mockup Generation

> ***Mục tiêu:*** *Agent tự sinh ra một bộ HTML mockup tương tác để user có thể preview UI trước khi implement, giảm thiểu rework sau này.*

```
flowchart TD
    A["📄 Phân tích requirement"] --> B["Xác định các màn hình UI"]
    B --> C["Thiết kế layout & navigation"]

    C --> SCREENS
    subgraph SCREENS ["🖥️ Screen Generation"]
        direction TB
        S1["Landing / Dashboard screen"]
        S2["List / Table screens"]
        S3["Detail / Form screens"]
        S4["Auth screens (login, register)"]
        S5["Error / Empty state screens"]
```

```
        S1 --- S2 --- S3 --- S4 --- S5
    end

    SCREENS --> D["Generate HTML + CSS + JS"]
    D --> E["Tạo navigation giữa các screens"]
    E --> F["Commit vào docs/mockup/"]
    F --> G["Link mockup URL vào issues"]
    G --> H{User review mockup}
    H -->|"Cần thay đổi"| I["AI update HTML"]
    I --> H
    H -->|"Approve"| J["✅ Mockup confirmed"]

    style SCREENS fill:#0d1117,stroke:#f0883e,color:#c9d1d9
```

Cấu trúc output:

```
docs/mockup/
├── index.html          # Navigation hub, danh sách tất cả screens
├── assets/
│   ├── style.css       # Global styles, design tokens
│   └── mock-data.js    # Placeholder JSON data
├── screens/
│   ├── dashboard.html
│   ├── user-list.html
│   ├── user-detail.html
│   ├── login.html
│   └── ...
└── README.md           # Hướng dẫn mở và review mockup
```

Quy tắc generate mockup:

Rule	Detail
Self-contained	Không cần server, mở trực tiếp bằng browser
Responsive	Mobile-first, breakpoints cho tablet & desktop
Placeholder data	Dùng realistic fake data, không để trống
Navigation	Sidebar/navbar liên kết đầy đủ các screens
Component consistent	Dùng chung design system (colors, fonts, spacing)
No external CDN	Inline styles/scripts để hoạt động offline

5.2 Phase 2 — Implement

> ***Mục tiêu:*** *AI Agent thực hiện implement từng issue, tracking progress, và xử lý user feedback.*

```
flowchart TD
    A(["🔨 Start Implementation"]) --> B["Lấy danh sách issues từ GitLab"]
    B --> C["Sắp xếp theo priority & dependency"]
```

```
    C --> D["Chọn issue tiếp theo có thể thực hiện"]

    D --> E["Update issue status: 🔄 In Progress"]
    E --> F["Tạo feature branch: feature/issue-N-title"]
    F --> G["Checkout branch"]

    G --> IMPL
    subgraph IMPL ["💻 AI Coding"]
        direction TB
        C1["Đọc issue description & requirements"]
        C2["Phân tích codebase hiện tại"]
        C3["Implement code changes"]
        C4["Viết unit tests"]
        C5["Run tests locally"]
        C6{Tests pass?}

        C1 --> C2 --> C3 --> C4 --> C5 --> C6
        C6 -->|No| C7["Fix bugs"]
        C7 --> C5
        C6 -->|Yes| C8["✅ Code ready"]
    end

    IMPL --> H["Commit với message: Implement #N - description"]
    H --> I["Push to remote branch"]
    I --> J["Update issue: Add progress comment"]

    J --> K["Update issue status: ✅ Done"]

    K --> L{User có comment mới?}
    L -->|"Yes"| M["Đọc & phân tích comment"]
    M --> N{Comment type?}
    N -->|"Bug report"| O["Fix bug → commit → push"]
    N -->|"Change request"| P["Update code → commit → push"]
    N -->|"Question"| Q["Reply comment trên GitLab"]
    O --> J
    P --> J
    Q --> J

    L -->|"No"| R{Còn issue chưa hoàn thành?}
    R -->|Yes| D
    R -->|No| S(["✅ Phase 2 Complete"])

    style IMPL fill:#0d1117,stroke:#3fb950,color:#c9d1d9
```

5.2.1 Branch Strategy

```
gitgraph
    commit id: "main"
    branch "feature/issue-1-project-setup"
    commit id: "setup project structure"
    commit id: "add configs"
```

```
    checkout main
    branch "feature/issue-2-database"
    commit id: "create schema"
    commit id: "add migrations"
    checkout main
    branch "feature/issue-3-api"
    commit id: "implement endpoints"
    commit id: "add validation"
    commit id: "fix review comment"
    checkout main
    branch develop
    merge "feature/issue-1-project-setup" id: "merge issue-1"
    merge "feature/issue-2-database" id: "merge issue-2"
    merge "feature/issue-3-api" id: "merge issue-3"
    checkout main
    merge develop id: "release merge"
```

5.2.2 Comment Handling Flow

```
flowchart LR
    A["🪝 Webhook: New Comment"] --> B["Parse comment content"]
    B --> C{Comment từ user?}
    C -->|No| D["Ignore - AI own comment"]
    C -->|Yes| E["Phân loại comment"]

    E --> F{Type}
    F -->|"🐛 Bug"| G["Create fix task"]
    F -->|"✏️ Change"| H["Update implementation"]
    F -->|"❓ Question"| I["Generate answer"]
    F -->|"👍 Approve"| J["Mark as resolved"]

    G --> K["AI implement fix"]
    H --> K
    I --> L["Post reply comment"]
    J --> M["Close discussion thread"]
    K --> N["Commit & push"]
    N --> L
```

5.3 Phase 3 — Review

> ***Mục tiêu:*** *Tạo Merge Request, xử lý review feedback cho đến khi MR được approve.*

```
flowchart TD
    A(["🔀 Start Review Phase"]) --> B["Tạo Merge Request"]

    B --> MR_CREATE
    subgraph MR_CREATE ["📝 MR Creation"]
        direction TB
        M1["Set title: Feature implementation - Sprint X"]
        M2["Generate MR description từ tất cả issues"]
```

```
        M3["Link related issues"]
        M4["Add reviewers"]
        M5["Set labels & milestone"]

        M1 --> M2 --> M3 --> M4 --> M5
    end

    MR_CREATE --> C["📢 Notify user: MR ready for review"]

    C --> D{User review MR}

    D -->|"❌ Request changes"| E["AI đọc review comments"]
    E --> F["Phân tích từng comment"]
    F --> G["Implement changes"]
    G --> H["Commit & push to MR branch"]
    H --> I["Reply to review comments"]
    I --> J["Re-request review"]
    J --> D

    D -->|"✅ Approve"| K["MR Approved"]
    K --> L(["✅ Phase 3 Complete"])

    style MR_CREATE fill:#0d1117,stroke:#a371f7,color:#c9d1d9
```

5.3.1 MR Description Template

```
## 🔀 Merge Request: [Project Name] Implementation

### Summary
[Tóm tắt tổng quan các thay đổi]

### Related Issues
- Closes #1 - Project Setup
- Closes #2 - Database Implementation
- Closes #3 - API Endpoints
- ...

### Changes
- ✅ [List of major changes]
- ✅ [Component A implemented]
- ✅ [Component B implemented]

### Testing
- [ ] Unit tests pass
- [ ] Integration tests pass
- [ ] Manual testing completed

### Documentation
- [ ] Architecture doc updated
- [ ] API doc updated
- [ ] README updated
```

```
### Screenshots / Evidence
[If applicable]
```

5.4 Phase 4 — Done

> ***Mục tiêu:*** *Merge code vào main branch, đóng issues, cleanup.*

```
flowchart TD
    A(["🏁 Start Done Phase"]) --> B["Merge MR vào target branch"]
    B --> C{Merge successful?}

    C -->|No: Conflict| D["AI resolve conflicts"]
    D --> E["Push resolved code"]
    E --> F["Re-request merge"]
    F --> C

    C -->|Yes| G["Đóng tất cả related issues"]
    G --> H["Delete feature branches"]
    H --> I["Update milestone status"]

    I --> REPORT
    subgraph REPORT ["📊 Final Report"]
        direction TB
        R1["Tổng số issues hoàn thành"]
        R2["Thời gian thực hiện"]
        R3["Số commits"]
        R4["Số review iterations"]
        R5["Summary of changes"]

        R1 --- R2 --- R3 --- R4 --- R5
    end

    REPORT --> J["Post report as comment on merged MR"]
    J --> K(["🎉 Project Complete!"])

    style REPORT fill:#0d1117,stroke:#16c79a,color:#c9d1d9
```

---

6. Sequence Diagrams

6.1 Full Lifecycle Sequence

```
sequenceDiagram
    actor User
    participant GL as GitLab
    participant ORCH as Orchestrator
    participant AI as Claude Code Agent
    participant FS as File System
```

```
Note over User,FS: ═══ Phase 1: INIT ═══

User->>GL: Push requirement.md
GL->>ORCH: Webhook: push event
ORCH->>AI: Parse requirement
AI->>FS: Read requirement file
AI->>AI: Analyze & plan
AI->>FS: Generate documents
AI->>GL: Create branch & commit docs
AI->>GL: Create issues - N issues
GL->>User: Notification: Issues created

loop Review Cycle
    User->>GL: Comment on issue
    GL->>ORCH: Webhook: comment event
    ORCH->>AI: Process feedback
    AI->>FS: Update documents
    AI->>GL: Update issue & commit
end

User->>GL: Approve plan via label/comment
GL->>ORCH: Webhook: approval event

Note over User,FS: ═══ Phase 2: IMPLEMENT ═══

loop For each Issue
    ORCH->>AI: Start issue N
    AI->>GL: Update issue: In Progress
    AI->>FS: Create feature branch
    AI->>FS: Implement code
    AI->>FS: Write tests
    AI->>FS: Run tests
    AI->>GL: Commit & push
    AI->>GL: Update issue: Done

    opt User Comment
        User->>GL: Comment on issue
        GL->>ORCH: Webhook: comment
        ORCH->>AI: Handle feedback
        AI->>FS: Update code
        AI->>GL: Push & reply
    end
end

Note over User,FS: ═══ Phase 3: REVIEW ═══

ORCH->>AI: Create MR
AI->>GL: Open Merge Request
GL->>User: Notification: MR ready

loop Review Cycle
    User->>GL: Review MR
```

```
        alt Changes requested
            GL->>ORCH: Webhook: MR review
            ORCH->>AI: Process review
            AI->>FS: Fix code
            AI->>GL: Push & reply comments
        else Approved
            User->>GL: Approve MR
        end
    end

    Note over User,FS: ═══ Phase 4: DONE ═══

    GL->>ORCH: Webhook: MR approved
    ORCH->>AI: Finalize
    AI->>GL: Merge MR
    AI->>GL: Close all issues
    AI->>GL: Delete branches
    AI->>GL: Post final report
    GL->>User: Notification: Complete 🎉
```

6.2 Webhook Event Handling

```
sequenceDiagram
    participant GL as GitLab
    participant WH as Webhook Server
    participant Q as Event Queue
    participant ORCH as Orchestrator
    participant AI as Claude Code

    GL->>WH: POST /webhook - event payload
    WH->>WH: Validate secret token
    WH->>WH: Parse event type

    alt Issue Comment Event
        WH->>Q: Enqueue: ISSUE_COMMENT
        Q->>ORCH: Dequeue & process
        ORCH->>ORCH: Check if from user not AI
        ORCH->>AI: Handle comment
        AI->>GL: Reply / update code
    else MR Review Event
        WH->>Q: Enqueue: MR_REVIEW
        Q->>ORCH: Dequeue & process
        ORCH->>AI: Handle review feedback
        AI->>GL: Update MR
    else Push Event
        WH->>Q: Enqueue: PUSH_EVENT
        Q->>ORCH: Dequeue & process
        ORCH->>ORCH: Check if requirement file changed
        ORCH->>AI: Re-analyze if needed
    else MR Merge Event
        WH->>Q: Enqueue: MR_MERGED
```

```
        Q->>ORCH: Dequeue & process
        ORCH->>AI: Run Done phase
        AI->>GL: Close issues & cleanup
    end
```

---

7. State Machine

7.1 Project State Machine

```
stateDiagram-v2
    [*] --> IDLE: Project created

    IDLE --> ANALYZING: Requirement received

    state INIT {
        ANALYZING --> GENERATING_DOCS: Analysis complete
        GENERATING_DOCS --> CREATING_ISSUES: Docs generated
        CREATING_ISSUES --> AWAITING_REVIEW: Issues created
        AWAITING_REVIEW --> UPDATING_PLAN: User requests changes
        UPDATING_PLAN --> AWAITING_REVIEW: Updates committed
        AWAITING_REVIEW --> PLAN_APPROVED: User approves
    }

    PLAN_APPROVED --> IMPLEMENTING: Start implementation

    state IMPLEMENT {
        IMPLEMENTING --> CODING: Pick next issue
        CODING --> TESTING: Code written
        TESTING --> CODING: Tests fail
        TESTING --> ISSUE_DONE: Tests pass
        ISSUE_DONE --> IMPLEMENTING: More issues
        ISSUE_DONE --> ALL_DONE: No more issues

        CODING --> HANDLING_FEEDBACK: User comment
        HANDLING_FEEDBACK --> CODING: Feedback processed
    }

    ALL_DONE --> REVIEW_PHASE: Create MR

    state REVIEW {
        REVIEW_PHASE --> MR_CREATED: MR opened
        MR_CREATED --> AWAITING_MR_REVIEW: Notify user
        AWAITING_MR_REVIEW --> FIXING_MR: Changes requested
        FIXING_MR --> AWAITING_MR_REVIEW: Fixes pushed
        AWAITING_MR_REVIEW --> MR_APPROVED: User approves
    }

    MR_APPROVED --> MERGING: Start merge

    state DONE {
```

```
        MERGING --> CLOSING_ISSUES: Merge successful
        MERGING --> RESOLVING_CONFLICTS: Merge conflict
        RESOLVING_CONFLICTS --> MERGING: Conflicts resolved
        CLOSING_ISSUES --> CLEANUP: Issues closed
        CLEANUP --> REPORTING: Branches deleted
    }

    REPORTING --> [*]: Complete
```

7.2 Issue State Machine

```
stateDiagram-v2
    [*] --> OPEN: Issue created

    OPEN --> IN_PROGRESS: AI starts working
    IN_PROGRESS --> IN_REVIEW: Code pushed

    IN_REVIEW --> IN_PROGRESS: User requests changes
    IN_REVIEW --> DONE: User approves

    DONE --> CLOSED: MR merged

    CLOSED --> [*]

    note right of OPEN
        Labels = status-open
    end note
    note right of IN_PROGRESS
        Labels = status-in-progress
    end note
    note right of IN_REVIEW
        Labels = status-in-review
    end note
    note right of DONE
        Labels = status-done
    end note
    note right of CLOSED
        Labels = status-closed
    end note
```

---

8. Data Model

8.1 Entity Relationship Diagram

```
erDiagram
    PROJECT ||--o{ ISSUE : contains
    PROJECT ||--o{ DOCUMENT : has
    PROJECT ||--|| MERGE_REQUEST : creates
    PROJECT {
```

```
    string id PK
    string name
    string gitlab_project_id
    string status
    string requirement_file_path
    datetime created_at
    datetime updated_at
}

ISSUE ||--o{ COMMENT : has
ISSUE ||--o{ COMMIT : linked_to
ISSUE {
    string id PK
    string gitlab_issue_id
    string project_id FK
    string title
    string description
    string status
    string branch_name
    int priority
    string labels
    datetime created_at
    datetime updated_at
}

COMMENT {
    string id PK
    string issue_id FK
    string gitlab_note_id
    string author
    string body
    string type
    boolean is_from_ai
    datetime created_at
}

DOCUMENT {
    string id PK
    string project_id FK
    string type
    string file_path
    string content_hash
    int version
    datetime created_at
    datetime updated_at
}

MERGE_REQUEST ||--o{ REVIEW_COMMENT : has
MERGE_REQUEST {
    string id PK
    string project_id FK
    string gitlab_mr_id
```

```
        string source_branch
        string target_branch
        string status
        string title
        string description
        datetime created_at
        datetime merged_at
    }

    REVIEW_COMMENT {
        string id PK
        string mr_id FK
        string gitlab_note_id
        string author
        string body
        string file_path
        int line_number
        boolean resolved
        datetime created_at
    }

    COMMIT {
        string id PK
        string issue_id FK
        string sha
        string message
        string branch
        datetime created_at
    }
```

8.2 Configuration Schema

```yaml
# config.yaml

# --- GitLab connection (dùng chung cho tất cả repos) ---
gitlab:
  url: "https://gitlab.company.com"
  token: "${GITLAB_ACCESS_TOKEN}"
  webhook_secret: "${WEBHOOK_SECRET}"

# --- Danh sách repositories ---
# Hỗ trợ nhiều repo trong cùng thư mục code
repositories:
  - name: "frontend"
    gitlab_project_id: 101
    local_path: "./repo-frontend"       # đường dẫn tương đối từ working dir
    type: "frontend"                    # frontend | backend | infra | fullstack
    tags: ["react", "typescript"]

  - name: "backend"
    gitlab_project_id: 102
```

```yaml
    local_path: "./repo-backend"
    type: "backend"
    tags: ["nodejs", "postgresql"]

  - name: "infra"
    gitlab_project_id: 103
    local_path: "./repo-infra"
    type: "infra"
    tags: ["docker", "k8s"]

# --- Agent settings ---
agent:
  model: "claude-sonnet-4-6"
  max_retries: 3
  timeout_seconds: 300
  mockup:
    enabled: true
    output_dir: "docs/mockup"
    framework: "vanilla"               # vanilla | tailwind | bootstrap

# --- Workflow settings (áp dụng cho tất cả repos) ---
workflow:
  auto_merge: false
  require_tests: true
  target_branch: "main"
  branch_prefix: "feature/"
  labels:
    init: ["phase:init", "ai-generated"]
    implement: ["phase:implement"]
    review: ["phase:review"]
    done: ["phase:done"]

notifications:
  enabled: true
  channels: ["gitlab-comment"]
```

---

9. GitLab API Integration

9.1 API Endpoints sử dụng

Operation	HTTP Method	Endpoint	Phase
Get project	`GET`	`/api/v4/projects/:id`	All
Create issue	`POST`	`/api/v4/projects/:id/issues`	Init
Update issue	`PUT`	`/api/v4/projects/:id/issues/:iid`	All
Close issue	`PUT`	`/api/v4/projects/:id/issues/:iid`	Done

List issue comments	`GET`	`/api/v4/projects/:id/issues/:iid/notes`	Implement
Add issue comment	`POST`	`/api/v4/projects/:id/issues/:iid/notes`	Implement
Create branch	`POST`	`/api/v4/projects/:id/repository/branches`	Implement
Delete branch	`DELETE`	`/api/v4/projects/:id/repository/branches/:branch`	Done
Create MR	`POST`	`/api/v4/projects/:id/merge_requests`	Review
Update MR	`PUT`	`/api/v4/projects/:id/merge_requests/:iid`	Review
Merge MR	`PUT`	`/api/v4/projects/:id/merge_requests/:iid/merge`	Done
List MR comments	`GET`	`/api/v4/projects/:id/merge_requests/:iid/notes`	Review
Add MR comment	`POST`	`/api/v4/projects/:id/merge_requests/:iid/notes`	Review
Create webhook	`POST`	`/api/v4/projects/:id/hooks`	Setup

9.2 Webhook Events cần lắng nghe

Event	Trigger	Action
`push_events`	Code pushed to repo	Check if requirement changed
`issue_events`	Issue created/updated/closed	Track issue status
`note_events`	Comment on issue/MR	Process user feedback
`merge_request_events`	MR created/updated/merged	Handle review flow

---

10. Tech Stack & Dependencies

```
graph TB
    subgraph "Runtime"
        NODE["Node.js / TypeScript"]
        HONO["Hono Framework"]
    end

    subgraph "AI"
        CLAUDE["Claude Code SDK"]
    end

    subgraph "GitLab"
        GL_API["GitLab REST API v4"]
        GL_HOOK["GitLab Webhooks"]
    end

    subgraph "Tools"
```

```
        GIT["Git CLI"]
        NPM["npm / pnpm"]
    end

    subgraph "Infrastructure"
        DOCKER["Docker - optional"]
        PM2["PM2 / systemd"]
    end

    NODE --> HONO
    NODE --> CLAUDE
    HONO --> GL_HOOK
    NODE --> GL_API
    NODE --> GIT
```

Category	Technology	Purpose
Runtime	Node.js + TypeScript	Core application runtime
Web Framework	Hono	Webhook server, lightweight & fast
AI Agent	Claude Code	Code generation, analysis, planning
Version Control	Git CLI	Branch management, commits, push
GitLab	REST API v4	Issue, MR, repository management
Process Manager	PM2	Keep orchestrator running
Container	Docker (optional)	Deployment isolation

---

11. Error Handling & Edge Cases

11.1 Error Handling Strategy

```
flowchart TD
    A["Error Occurred"] --> B{Error Type}

    B -->|"GitLab API Error"| C{Status Code}
    C -->|401/403| D["🔑 Auth error - Alert user"]
    C -->|404| E["Resource not found - Log & skip"]
    C -->|429| F["⏳ Rate limited - Retry with backoff"]
    C -->|500+| G["Server error - Retry 3 times"]

    B -->|"AI Error"| H{AI Error Type}
    H -->|"Timeout"| I["⏰ Retry with longer timeout"]
    H -->|"Invalid output"| J["🔄 Re-prompt with clarification"]
    H -->|"Context too large"| K["✂️ Chunk and retry"]

    B -->|"Git Error"| L{Git Error Type}
    L -->|"Merge conflict"| M["🔀 Auto-resolve or alert user"]
```

```
    L -->|"Push rejected"| N["📤 Pull & rebase - retry push"]
    L -->|"Branch exists"| O["📁 Use existing or rename"]

    B -->|"Network Error"| P["🌐 Retry with exponential backoff"]

    D --> Q["Post error to GitLab issue comment"]
    G --> Q
    I --> Q
    M --> Q
    P --> Q
```

11.2 Edge Cases

Edge Case	Handling Strategy
User deletes an issue mid-process	Detect via webhook, skip issue, log warning
Requirement file changes during implementation	Pause, re-analyze diff, create new issues if needed
GitLab server unavailable	Queue operations, retry with exponential backoff
AI generates invalid code	Run linting/tests, auto-fix or request user help
Merge conflicts	Auto-resolve simple conflicts, escalate complex ones
Multiple users commenting simultaneously	Process comments in order, use locks
Large codebase exceeding context	Chunk files, use targeted analysis
Circular issue dependencies	Detect cycles, alert user, suggest resolution

---

12. Security Considerations

Concern	Mitigation
GitLab Token	Store in environment variables, never in code
Webhook Secret	Validate `X-Gitlab-Token` header on every request
Code Execution	Sandbox AI-generated code, review before merge
Access Control	Use minimum-privilege GitLab token (API scope only)
Data Privacy	Don't log sensitive data, sanitize AI inputs/outputs
Rate Limiting	Implement rate limiting on webhook endpoint
Input Validation	Validate all webhook payloads before processing

---

13. Docker Deployment

13.1 Container Architecture

```
graph TB
    subgraph "Docker Host"
        subgraph "docker-compose stack"
            ORCH_C["🎯 orchestrator\n(Node.js + Hono)"]
            REDIS_C["🗄️ redis\n(event queue)"]
            AGENT_C["🤖 agent-runner\n(Claude Code)"]

            ORCH_C -->|"enqueue/dequeue"| REDIS_C
            ORCH_C -->|"spawn task"| AGENT_C
        end

        VOL_CODE["📁 Volume: /workspace\n(git repos)"]
        VOL_CFG["⚙️ Volume: /config\n(config.yaml)"]
        VOL_LOGS["📋 Volume: /logs"]

        AGENT_C <-->|"read/write code"| VOL_CODE
        ORCH_C <-->|"read config"| VOL_CFG
        ORCH_C <-->|"write logs"| VOL_LOGS
    end

    GITLAB["🦊 Self-hosted GitLab"] -->|"webhooks"| ORCH_C
    ORCH_C -->|"GitLab API"| GITLAB
```

13.2 Dockerfile

```
# Orchestrator Service
FROM node:22-alpine AS base
WORKDIR /app

FROM base AS deps
COPY package.json pnpm-lock.yaml ./
RUN corepack enable pnpm && pnpm install --frozen-lockfile

FROM base AS builder
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN pnpm build

FROM base AS runner
ENV NODE_ENV=production

# Cài git và claude CLI
RUN apk add --no-cache git openssh-client
RUN npm install -g @anthropic-ai/claude-code

COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./
```

```
EXPOSE 3000

CMD ["node", "dist/index.js"]
```

13.3 docker-compose.yml

```yaml
services:
  orchestrator:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: ai-agent-orchestrator
    restart: unless-stopped
    ports:
      - "3000:3000"          # Webhook endpoint
    environment:
      - NODE_ENV=production
      - GITLAB_ACCESS_TOKEN=${GITLAB_ACCESS_TOKEN}
      - WEBHOOK_SECRET=${WEBHOOK_SECRET}
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - REDIS_URL=redis://redis:6379
    volumes:
      - ./config.yaml:/app/config.yaml:ro
      - workspace:/workspace         # shared git repos
      - logs:/app/logs
    depends_on:
      redis:
        condition: service_healthy
    networks:
      - agent-net

  redis:
    image: redis:7-alpine
    container_name: ai-agent-redis
    restart: unless-stopped
    command: redis-server --save 60 1 --loglevel warning
    volumes:
      - redis-data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 3
    networks:
      - agent-net

volumes:
  workspace:
    driver: local
    driver_opts:
```

```yaml
      type: none
      o: bind
      device: ${WORKSPACE_PATH:-./workspace}  # thư mục chứa các git repos
  redis-data:
  logs:

networks:
  agent-net:
    driver: bridge
```

13.4 Environment Variables

Tạo file `.env` ở root:

```
# .env
# GitLab
GITLAB_ACCESS_TOKEN=glpat-xxxxxxxxxxxxxxxxxxxx
WEBHOOK_SECRET=your-webhook-secret-here

# Anthropic
ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxxxxxxxxxx

# Workspace — thư mục chứa tất cả git repos
WORKSPACE_PATH=/path/to/your/repos

# Optional
PORT=3000
LOG_LEVEL=info
```

13.5 Khởi chạy

```bash
# 1. Clone project
git clone https://gitlab.company.com/ai-agent-coding.git
cd ai-agent-coding

# 2. Cấu hình
cp .env.example .env
# Chỉnh sửa .env với các credentials thực
nano .env

# 3. Chuẩn bị workspace (chứa các git repos cần làm việc)
mkdir -p ./workspace
git clone https://gitlab.company.com/your-project/frontend.git ./workspace/repo-
frontend
git clone https://gitlab.company.com/your-project/backend.git ./workspace/repo-
backend

# 4. Cấu hình config.yaml (khai báo repositories)
cp config.example.yaml config.yaml
nano config.yaml
```

```
# 5. Build & run
docker compose up -d --build

# 6. Kiểm tra logs
docker compose logs -f orchestrator

# 7. Đăng ký webhook trên GitLab
# Trỏ webhook URL đến: http://your-server:3000/webhook
```

13.6 Health Check & Monitoring

```
# Kiểm tra trạng thái containers
docker compose ps

# Xem logs real-time
docker compose logs -f

# Restart orchestrator (sau khi update config)
docker compose restart orchestrator

# Dừng toàn bộ stack
docker compose down

# Dừng và xóa volumes (reset hoàn toàn)
docker compose down -v
```

Endpoint	Method	Description
`GET /health`	GET	Health check status
`GET /status`	GET	Current workflow state, active issues
`POST /webhook`	POST	GitLab webhook receiver
`POST /trigger`	POST	Manually trigger a workflow phase

> ***Next Steps:***
>
> 1. *Setup project structure theo architecture*
> 2. *Implement Orchestrator service với Hono framework*
> 3. *Tích hợp Claude Code SDK*
> 4. *Implement GitLab API client*
> 5. *Setup webhook server*
> 6. *Implement Multi-repo Manager*
> 7. *Implement Mockup Generator*
> 8. *Setup Docker deployment*
> 9. *Testing end-to-end workflow*